

MA388 Sabermetrics: Lesson 6

How many runs is a win? - Part I

LTC Jim Pleuss

Purpose

Last class we focused on generating slides using Quarto, and we explored ways to scale up your production to generate a set of slides for any combination of teams, years, etc.

What are some benefits of this capability?

- Easily generate products at scale using a proven analysis pipeline.
- You can email your generated product. There's no web hosting.

What are some limitations?

- If your product doesn't answer the reader's question, the reader has no way to refine the product to answer their question of interest.
- The reader has very limited opportunity to interact with the data. The data doesn't automatically travel with the static document you've created (possibly good or bad).

Shiny Apps!

A Shiny app is fundamentally different. Now you'll host a small (though it could be quite large) application on a web server, and people will visit it via a published URL. It's pretty simple:

- Allow the user to select some permissible values of different parameters you'll use to control any analysis you present.
- Lay out a user interface (UI) and define server functions that turn user selections into tables, plots, etc.

Starting Example

Step 1: File -> New File -> Shiny Web App

Give the app a name with no spaces. I called mine “BattingLeaders” and selected the “Single File” option (default) and specified a directory where I wanted the app to live. I now have a folder in this locaiton called “BattingLeaders” with a single file titled “app.R” inside.

Step 2: Press the “Run App” button near the top right of the app editor. Drag the slider to different values and see what happens.

All the basics are here - a title, an input widget, and a reactive plot that changes as you change the input slider. This is basically it for Shiny Apps – you mix static content, user inputs, and reactive content.

Step 3: Explore the code that produced this app.

You should see a section called the `ui` that has elements you can connect to what you see in the app - a layout, some panels, a slider input, and a plot output.

Now look at the `server` section. There’s a single `output` element named `distplot` that is created in the server and made visible in the `ui` section as an argument to the `plotOutput` function. Notice that the plot code contained in this element would work anywhere with one exception – it is reliant on knowing what `input$bins` equals. You can guess where it gets that value – it’s from the `sliderInput` object named “bins” (the first element of an input object definition is its name/ID).

Step 4: Tweak this app to do something else.

- Add static text.

After the `titlePanel` line in the `ui`, add this as a new line (including the comma):

```
h6("This is some really helpful explanation of what you're looking  
at."),
```

Run the app again to see what happened.

- Add an input.

Put a comma at the end of the `sliderInput` definition after `value=30`). On the next line, add this text object:

```
h6("I couldn't think of anything good to add here, so I'm switching  
data sets:"), (include the comma)
```

...followed by this new input object:

```
radioButtons("cylinders",
             label = h3("Select a number of cylinders to explore in mtcars:"),
             choices = list("4 cylinders" = 4,
                           "6 cylinders" = 6,
                           "8 cylinders" = 8),
             selected = 4)
```

How do you learn what input widgets are available and how to employ them? Visit the [Shiny Widget Gallery](#).

Run the app again to see what happened.

- Add an output.

Let's add a plot to the app that does something with `mtcars` and your input selection. Copy and paste the `output$distPlot` code block immediately underneath the existing code block (no need for a comma between the two plot blocks). Call this second plot block `output$distPlot2` and add a second `plotOutput` object to the `ui` section like this:

```
mainPanel(
  plotOutput("distPlot"),
  plotOutput("distPlot2"),
)
```

Run the app again to see what happened. We should see our cylinders radio buttons and a second inter-eruption distribution plot. It's time to connect these objects. First, add `library(tidyverse)` to the top of the app by the `library(shiny)` call. Next, change `distPlot2` as follows:

```
output$distPlot2 <- renderPlot({
  mtcars |>
    filter(cyl == input$cylinders) |>
    ggplot(aes(x = mpg)) +
    geom_histogram(bins = 6) +
    labs(x = 'MPG',
         title = paste('Histogram of MPG for vehicles with',
                       input$cylinders,
                       "cylinders"))
})
```

That's it! This basic knowledge – plus tons of creativity and trial and error – is what you need to make a much cooler, much more complicated app.

Challenge Problem

Make a Shiny App that allows the user to select a starting year and ending year between 1970 and 2023 as well as a choice of batting stats (start with batting average and home run total).

Display a histogram of all players' stats during the time period of interest, showing only the histogram of the selected batting stat.